

Bases de Données Avancées
Migration MongoDB
RAPPORT 2

Felipe Mateus Falcão Barreto

1. Définition de cette étape du projet

Cette étape, consacrée à MongoDB, avait pour objectif de comprendre les différences entre les bases de données relationnelles (SQL) et non relationnelles (NoSQL), d'utiliser le pipeline d'agrégation de MongoDB et de comparer leurs performances. Le serveur MongoDB et la bibliothèque PyMongo ont été utilisés pour configurer l'environnement.

Ensuite, les collections ont été migrées à l'aide du script `migrate_flat.py`, qui a créé une collection MongoDB pour chaque table existante dans la base de données SQLite, a extrait les données directement de la base de données relationnelle (sans utiliser de fichiers CSV), a inséré les documents dans MongoDB et a finalement vérifié que le nombre d'enregistrements migrés était correct.

2. — Modèle de document structuré (schéma JSON)

Le modèle de document structuré adopté dans ce projet repose sur la dénormalisation des données initialement stockées dans une base de données relationnelle SQLite, selon les principes du système NoSQL orienté documents MongoDB. Au lieu de répartir l'information dans plusieurs tables interconnectées par des clés étrangères, il a été décidé de regrouper toutes les données pertinentes associées à un film dans un seul document. Ce document, stocké dans la collection `movies_complete`, représente une vue complète et autonome de l'entité `movie`.

Le schéma JSON structuré comprend les attributs de base des films, tels que l'identifiant `mid`, le titre, l'année de sortie et la durée, ainsi que des tableaux pour des informations plus complexes. Les genres sont stockés sous forme de tableaux de chaînes de caractères, ce qui permet des requêtes directes sans jointure. Les notes sont encapsulées dans un sous-document (`rating`) contenant la note moyenne et le nombre de votes, facilitant ainsi l'accès à ces valeurs.

De plus, le document comprend des listes de réalisateurs, d'acteurs, de scénaristes et de titres. Chaque élément est représenté par un tableau de sous-documents, contenant des identifiants de personnes, des noms et des attributs spécifiques. Cette structure reflète un modèle orienté requêtes, dont l'objectif principal est de réduire le nombre d'accès à la base de données et d'éliminer le besoin de multiples jointures, fréquentes dans le modèle relationnel.

3. — Les 9 requêtes MongoDB équivalentes

Dans cette étape, 9 requêtes MongoDB ont été implémentées, reproduisant la même logique d'extraction et d'analyse de données que les requêtes SQL développées lors de la phase relationnelle. L'objectif était de permettre une comparaison directe entre les approches SQL et NoSQL. Des opérateurs tels que `$match`, `$lookup`, `$group`, `$project`, `$sort` et `$limit` ont été utilisés pour développer le pipeline.

- `filmsByActor`

Cette requête renvoie la liste des films dans lesquels un acteur a joué. L'acteur est identifié par son nom, et les données sont croisées avec les collections `principals` et `movies` pour obtenir les titres, les années et le type de participation. Les résultats sont triés par année de sortie.

- `bestFilmsFilter`

Cette requête sélectionne les N meilleurs films d'un genre spécifique sur une période donnée. Les collections genres, movies et ratings sont combinées, ce qui vous permet de filtrer par période et de trier les films par note moyenne et nombre de votes.

- `charactersByActor`

Cette requête identifie les acteurs ayant interprété plusieurs personnages dans un même film. Les données sont regroupées par acteur et par film, et seuls les cas comportant plusieurs rôles sont conservés. Les informations relatives au nom et au titre sont obtenues par des jointures supplémentaires.

- `filmsTogether`

Cette requête analyse les réalisateurs avec lesquels un acteur a travaillé et le nombre de films qu'ils ont tournés ensemble. Elle croise les collections principaux, directors et persons en regroupant les résultats par réalisateur et en comptabilisant les collaborations.

- `bestMoviesMetrics`

Esta consulta calcula a nota média e o número de filmes por gênero. Apenas gêneros com nota média superior a um limite e quantidade mínima de filmes são retornados, permitindo identificar os gêneros mais bem avaliados.

- `bestByDecade`

Cette requête analyse la carrière d'un acteur au fil du temps. Les films sont regroupés par décennie, ce qui permet de calculer le nombre de productions et la note moyenne pour chaque période de sa carrière.

- `moviesPodium`

Cette requête établit un classement des meilleurs films de chaque genre. Elle utilise un système de notation interne pour sélectionner les K films les mieux notés par genre, en tenant compte de la note moyenne et de la popularité.

- `famousByMovie`

Cette demande vise à identifier les acteurs ayant participé à des films ayant reçu un grand nombre de votes. L'objectif est de mettre en valeur leur participation à des productions à succès et à forte visibilité.

- `top_actors_by_genre`

Cette requête détermine les acteurs les mieux notés dans un genre donné. Pour chaque acteur, le nombre d'apparitions et la note moyenne des films sont calculés, ce qui permet d'établir un classement selon sa performance dans le genre.

4. — Tableau comparatif SQL vs MongoDB

SQL et MongoDB représentent deux paradigmes de bases de données différents. SQL utilise des modèles relationnels, avec des tables normalisées, des clés primaires et étrangères, tandis que MongoDB est une base de données NoSQL, basée sur des documents JSON, permettant des structures plus flexibles et hiérarchiques.

Fonctionnalité

SQL

MongoDB

Modèle de données	Relationnel (tables, lignes, colonnes)	Document (JSON/BSN structuré)
Flexibilité du schéma	Rigide, les changements nécessitent des migrations.	Dynamique, s'adapte facilement aux nouveaux domaines.
Requêtes	SQL standard (SELECT, JOIN, GROUP BY)	Cadre d'agrégation et requêtes BSN
Performances pour les jointures complexes	Peut être lente à plusieurs points de jonction.	Plus rapide sur les données intégrées/dénormalisées
Évolutivité	Vertical (plus de processeur/RAM)	Horizontal (partitionnement natif)

4.1 Temps d'exécution

Após a migração para documentos estruturados, observou-se:

- Les requêtes qui nécessitaient plusieurs jointures entre les tables du modèle SQL (ou plusieurs requêtes dans les collections MongoDB) sont désormais exécutées beaucoup plus rapidement grâce à l'agrégation des données dans un seul document.
- Des opérations comme `filmsByActor` ou `bestFilmsFilter` permettent de réduire considérablement le temps d'exécution, notamment lors du traitement de grands volumes de données. En effet, le modèle de document structuré permet de regrouper les informations connexes au sein d'un seul document, ce qui diminue le nombre de requêtes nécessaires pour rassembler l'ensemble des données.
- Pour les requêtes simples, les temps de réponse restent très faibles, comparables à ceux observés dans les bases de données relationnelles bien optimisées. En revanche, les agrégations plus complexes, impliquant de multiples relations ou calculs, tirent parti de la structure hiérarchique des documents JSON/BSN, évitant ainsi les multiples requêtes entre les collections et réduisant la latence globale.

4.2 Complexité du code

Dans le modèle plat, l'obtention d'informations complètes nécessite de multiples requêtes enchaînées sur différentes collections, combinées à une logique supplémentaire pour fusionner les résultats. Cela accroît la complexité du code, le rendant plus difficile à maintenir et augmentant le risque d'erreurs ou d'incohérences lors de l'intégration des données.

Le modèle JSON structuré permet d'utiliser des pipelines d'agrégation plus compacts et efficaces, en concentrant toute la logique de jointure, de filtrage et de transformation directement dans MongoDB. Cela réduit la quantité de code réparti entre l'application et la base de données, rendant les flux de données plus lisibles.

4.3 Avantages et inconvénients

Avantages de MongoDB structuré:

- Réduction du nombre de requêtes pour obtenir des données liées.
- Structure facilement adaptable à de nouveaux champs et types de données.

- Performances améliorées lors de la lecture de documents complets.

Inconvénients/Limitations :

- Les structures complexes peuvent générer des documents volumineux, ce qui affecte la vitesse d'écriture.
- Les agrégations complexes nécessitent un apprentissage du framework.

5. — Analyse documents plats et structurés

5.1 Modèle plat

Dans ce modèle, chaque type d'information est stocké dans des collections distinctes. Pour obtenir toutes les données relatives à un film, il est nécessaire d'exécuter plusieurs requêtes et de fusionner les résultats dans l'application.

Avantages :

- Structure simple de chaque collection.
- Mise à jour aisée des données isolées sans impacter les autres documents.

Inconvénients:

- L'enchaînement de nombreuses requêtes augmente le temps d'exécution, notamment avec de grands volumes de données.
- Code plus complexe, sujet aux erreurs lors de la fusion des résultats.
- Surcharge d'E/S due aux multiples appels à la base de données.

5.2 Modèle structuré:

Les données relatives à un film sont consolidées dans un seul document JSON, contenant des informations sur les genres, les notes, les réalisateurs, les acteurs, les personnages, les scénaristes et les titres. La construction de ce document utilise les pipelines d'agrégation de MongoDB pour intégrer toutes les informations des collections associées dans un seul enregistrement.

Avantages:

- Réduction du nombre de requêtes: les données d'un film peuvent être récupérées en une seule lecture du document.
- Il n'y a pas de multiples jointures manuelles, (facilite les requêtes complexes).

Inconvénients / limitations:

- Les documents volumineux peuvent engendrer des coûts d'écriture et une consommation de mémoire plus élevés.
- La mise à jour partielle de champs imbriqués peut s'avérer plus complexe et nécessiter des pipelines supplémentaires.
- La courbe d'apprentissage est plus abrupte, car l'utilisation du framework d'agrégation est plus complexe.

Le modèle structuré offre des gains considérables en termes de performances et de maintenance, notamment lorsque les opérations avec des données sont récurrentes. Le modèle plat reste utile en cas de données faiblement corrélées, mais présente des limitations lors d'un accès intensif à des données intégrées.